

Atividade Avaliativa Prática 4/6

Hello World: aproximação do seno no ESP32-S3

Estudante: Rafaela Santana

Disciplina: IA embarcada e modelos compactos

1. Objetivo e reprodução

A atividade utiliza o TensorFlow Lite Micro para executar uma rede neural no ESP32-S3. O modelo recebe um número no intervalo de zero a aproximadamente 2π e estima o seu seno. Trata-se de inferência local: o treinamento ocorre no computador, enquanto o microcontrolador executa o modelo convertido.

O projeto utiliza um modelo quantizado para aproximar o seno, com medições de erro e verificações de execução. Não foi realizado novo treinamento nesta atividade.

Para reproduzir, deve-se ativar o ambiente ESP-IDF, executar `idf.py set-target esp32s3` e `idf.py build`, abrir a pasta no VS Code e iniciar **Wokwi: Start Simulator**. O `diagram.json` representa a placa e a conexão UART com o monitor serial. O `wokwi.toml` aponta para `build/flasher_args.json`, permitindo carregar aplicação, bootloader e tabela de partições. Os comandos completos estão no `README.md`.

2. Captura da execução no Wokwi

Pendente de inclusão: inserir aqui uma captura própria do Wokwi executando esta versão, com a placa, o monitor serial e a mensagem `CICLO CONCLUIDO` visíveis.

Não foram confirmadas nesta etapa a execução no Wokwi nem as métricas obtidas no simulador. Essa pendência precisa ser resolvida antes da entrega para atender ao item 2 do enunciado.

3. Análise do código

Em `main.cc`, a função `app_main()` inicializa a aplicação e repete a inferência com uma espera de 250 ms entre iterações. Em `main_functions.cc`, `GetModel()` acessa o modelo armazenado no programa, verifica-se a versão do esquema e cria-se um `MicroInterpreter`. O registro usa apenas `FullyConnected`, evitando solicitar operadores desnecessários. A arena estática de 4.096 bytes, alinhada a 16 bytes, fornece memória para os tensores e estruturas internas; seu consumo efetivo é informado no início da execução.

A entrada e a saída são `int8`. Para representar uma entrada real, o programa calcula `q = round(x / escala) + zero_point` e limita o resultado ao intervalo de -128 a 127. Após a inferência, recupera a saída real por `y = (q - zero_point) × escala`. A quantização permite representar cada valor desses tensores em um byte, mas introduz arredondamento; ela não garante, por si só, que todo o firmware fique quatro vezes menor ou mais rápido.

Um indicador de inicialização evita impedir o acesso aos tensores quando a inicialização falha. O programa verifica o tipo, o tamanho escalar e as escalas dos tensores antes da inferência.

Em `output_handler.cc`, a previsão da rede é comparada com `sin(x)`. A função matemática é somente uma referência de avaliação, sem substituir o resultado do modelo. São exibidos o erro absoluto de cada amostra e, ao fim de 40 pontos, o erro absoluto médio (MAE), a raiz do erro quadrático médio (RMSE) e o maior erro. O RMSE dá maior peso aos desvios grandes. As estatísticas são reiniciadas a cada ciclo.

4. Modificações e observações

A amostragem utiliza 40 pontos, com intervalo de 250 ms, mantendo cerca de dez segundos de espera acumulada por ciclo. O domínio continua em $[0, 2\pi)$, sem repetir o ponto final. A saída serial passou a permitir uma comparação direta entre previsão e referência, facilitando identificar regiões em que a aproximação é menos precisa.

A conferência binária mostrou que `main/model.cc` contém exatamente os mesmos 3.344 bytes do arquivo `notebooks/hello_world_int8.tflite`. O script `scripts/converter_modelo.py` também permite regenerar o array com os nomes de símbolos exigidos pelo firmware.

Os erros medidos nos 40 pontos caracterizam essa varredura determinística. Não correspondem a uma avaliação em um novo dataset independente, nem permitem afirmar precisão para entradas fora do domínio. A arena maior representa uma reserva de memória; o consumo real deve ser consultado no monitor serial.

5. Conclusão e escopo

A análise evidencia as etapas de uma aplicação TinyML: exportação do modelo, armazenamento no firmware, alocação de memória, quantização, inferência e interpretação da saída. As modificações tornam o comportamento mais observável e evitam acesso aos tensores após falhas de inicialização. A conclusão experimental depende da captura e da leitura dos resultados da simulação.

Esta versão contempla o desenvolvimento da parte obrigatória, com a evidência visual ainda pendente. Não foi implementado o extra de novo sensor, novo dataset e treinamento para uma nova funcionalidade; alterações de apresentação ou amostragem não atendem a esse requisito.

Referências

- Wokwi, configuração de projetos ESP-IDF: <https://docs.wokwi.com/vscode/project-config>.
- TensorFlow Lite Micro: <https://github.com/tensorflow/tflite-micro>.